

create model [each rank gets $\frac{1}{\text{world-size}}$ of parameters]

forward pass

compute activation (batch-size $\times$ local-num-dim)

Allocate memory for activations (world-size $\times$ batch-size $\times$ local-num-dim)

send activations via all gather

concatenate them to get batch-size $\times$ Num-dim

↓ # backward pass

def pipeline_parallelism():

Sharding strategy: each rank gets

• a subset of layers, transfer all data/activation

~~def pipeline_parallelism_main():~~

data = generate_sample_data()

spawn(pipeline_parallelism_main, world_size=2, data=data, num_layers=4,
→ launch 2 = world-size process to execute two

↓ p def pipeline_parallelism_main(): ~~pipeline_parallelism_main()~~ {num_micro_batches=4}

use all the data

split up layers

each rank gets a ~~rank~~ subsets of layers

forward pass

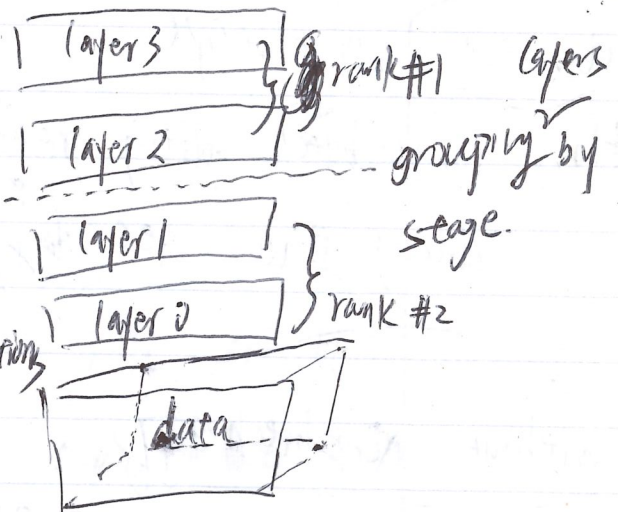
↓ secondary/turnover ~~optimization~~ ^{parallelization} for speed.

Break up into mini/micro batches to minimize bubble

Allocate memory for activation

for x in micro_batches:

Get activations from previous banks



waiting time

↑

bubble

part #2 $\left[\begin{smallmatrix} \text{rank} \\ \text{layers} \end{smallmatrix} \right]$ layers assigned to this rank
to the next rank (asynchronously)

def d overlapping communication/computation to eliminate
Interleave ~~for~~ forward and backward passes) pipeline bubbles
=> backward pass: ----

clean-up: ----

- 1: Receiving gradients from the next rank
- 2: computing ~~for~~ the local gradients with activations
- 3: sending gradients to the previous rank

Question: ask chatGPT

How micro-batch splitting reduce the rank idle time
during the secondary ^{pipeline} parallelization forward/backward passes?

without micro-batching

Time step	GPU0 (Rank=0)	GPU1 (Rank=1)
t=1	compute layers 1-2	Idle state
t=2	idle status	compute layers 3-4

=> only one GPU works at a time - total utilization $\approx 50\%$
this idle time is the pipeline bubble - the other devices wait for data

with micro-batching